`~/Downloads/design_example.md`

Design of Tic Tac Tos

Group members: Mahbod Seihoon (581442), Sergej Savić (593728)

This document gives the high-level design of a tic tac toe game implementation. It specifies the data structures that are used, the functions that operate on those data structures, and describes how the different player interact with components to implement the game of tic-tac-toe.

Game overview

Ultimate Tic Tac Toe is played on a 3x3 grid of smaller Tic Tac Toe boards (making a 9-board system). Players take turns placing X or O in a cell of the current board; the position of that move determines which small board the opponent must play in next. A player wins a small board by making three in a row; the overall game is won by winning three small boards in a row.

Design components

`The game state is represented using the following entities:`

Tile: Represents a single cell. Each tile can be empty, X, or O. **SmallGame**: Represents a 3×3 Tic-Tac-Toe board made of `Tile`s. **BigGame**: Represents the Ultimate board, a 3×3 grid of `SmallGame`s. **Round**: Represents a full game round, handles turn order and move flow.

All entities are implemented using the **opaque data type pattern**.
Only interfaces are exposed in header files.

Note:

To keep the documentation concise and easy to read, all interfaces are presented in the header (.h) files in this document.

In the actual project implementation, all data structure definitions and function implementations are placed in the corresponding .c files, following the Opaque Type Pattern and proper encapsulation principles.

Data Structures and enumerations

Tile

tile.h

```
#ifndef TILE_H
#define TILE_H

typedef enum
{
    TILE_EMPTY,
    TILE_X,
    TILE_O
} tile_state_t;

typedef struct tile tile_t;

tile_t *tile_create(void);
void tile_destroy(tile_t *tile);
void tile_set_state(tile_t *tile, tile_state_t state);
tile_state_t tile_get_state(const tile_t *tile);

#endif
```

tile.c

```
#include "tile.h"

struct tile
{
    tile_state_t state;
};
```

Small Game

small_game.h

```
#ifndef SMALL_GAME_H
#define SMALL_GAME_H

#include "tile.h"

typedef enum
{
    SMALL_GAME_ONGOING,
    SMALL_GAME_X_WON,
    SMALL_GAME_O_WON,
    SMALL_GAME_DRAW
} small_game_state_t;

typedef struct small_game small_game_t;

small_game_t *small_game_create(void);
void small_game_destroy(small_game_t *game);
small_game_state_t small_game_get_state(const small_game_t *game);

#endif
```

small_game.c

```
#include "small_game.h"

struct small_game
{
    tile_t *tiles[3][3];
    small_game_state_t state;
};
```

Big Game

big_game.h

```
#ifndef BIG_GAME_H
#define BIG_GAME_H

#include "small_game.h"
```

```
typedef enum
{
    BIG_GAME_ONGOING,
    BIG_GAME_X_WON,
    BIG_GAME_O_WON,
    BIG_GAME_DRAW
} big_game_state_t;

typedef struct big_game big_game_t;

big_game_t *big_game_create(void);
void big_game_destroy(big_game_t *game);
big_game_state_t big_game_get_state(const big_game_t *game);

#endif
```

big_game.c

```
#include "big_game.h"

struct big_game
{
    small_game_t *boards[3][3];
    int forced_row;
    int forced_col;
    big_game_state_t state;
};
```

Round

round.h

```
#ifndef ROUND_H
#define ROUND_H

#include "big_game.h"

typedef enum
{
    PLAYER_X,
    PLAYER_O
```

```
} player_t;

typedef struct round round_t;

round_t *round_create(void);
void round_destroy(round_t *round);
player_t round_get_current_player(const round_t *round);

int round_play_move(
    round_t *round,
    int board_row,
    int board_col,
    int cell_row,
    int cell_col);

#endif
```

round.c

```
#include "round.h"

struct round
{
    big_game_t *game;
    player_t current_player;
    int round_over;
};
```